

# TRPO 实验报告

## 0. 实验概况

本次实验实现并测试 TRPO (Trust Region Policy Optimization)，完成 `compute_advantages_from_traj`、`compute_advantages_from_rollout` 和 `_update_policy` 三部分代码补全，并在以下三个 MuJoCo Gymnasium v5 环境中进行实验：

- `HalfCheetah-v5`
- `Hopper-v5`
- `Walker2d-v5`

实验比较了两类采样方式：

- `rollout`
- `trajectory`

以及两类 advantage 设置：

- `advan_norm=true`
- `advan_norm=false`

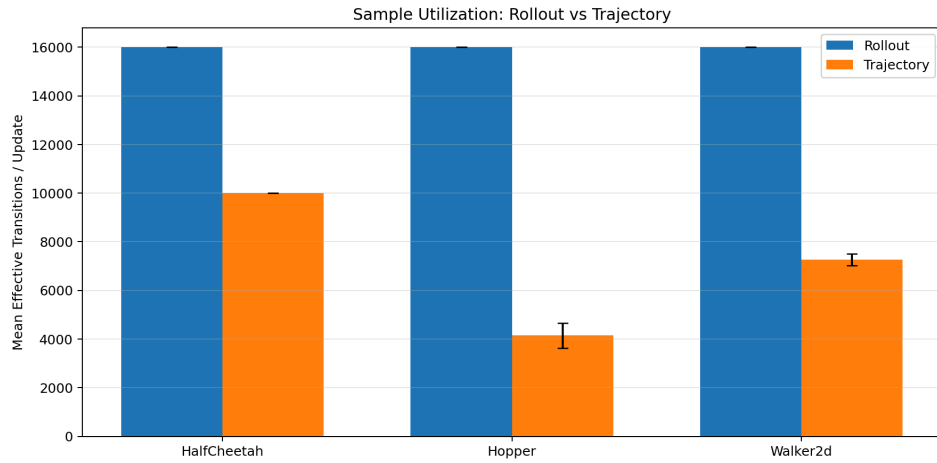
每组配置使用 3 个随机种子，共完成 36 组实验。

## 0.1 实验结果总表

| Env            | Mode       | Advan Norm | Mean Best Reward | Std   | Mean Final Reward | Mean KL  | Mean Efficiency |
|----------------|------------|------------|------------------|-------|-------------------|----------|-----------------|
| HalfCheetah-v5 | rollout    | False      | 4382.4           | 452.0 | 4344.7            | 7.89e-03 | 16000           |
| HalfCheetah-v5 | rollout    | True       | 4365.6           | 471.2 | 4303.3            | 7.77e-03 | 16000           |
| HalfCheetah-v5 | trajectory | False      | 3833.6           | 731.8 | 3754.7            | 8.03e-03 | 10000           |
| HalfCheetah-v5 | trajectory | True       | 4028.6           | 452.8 | 3910.2            | 7.88e-03 | 10000           |
| Hopper-v5      | rollout    | False      | 3567.5           | 71.2  | 1359.8            | 5.76e-03 | 16000           |
| Hopper-v5      | rollout    | True       | 3572.6           | 65.7  | 1419.4            | 5.67e-03 | 16000           |
| Hopper-v5      | trajectory | False      | 3452.8           | 72.7  | 2176.9            | 7.37e-03 | 4349            |
| Hopper-v5      | trajectory | True       | 3587.2           | 100.4 | 1589.7            | 7.14e-03 | 3934            |
| Walker2d-v5    | rollout    | False      | 4515.7           | 160.6 | 4348.9            | 8.73e-03 | 16000           |
| Walker2d-v5    | rollout    | True       | 4718.7           | 380.4 | 4317.8            | 8.71e-03 | 16000           |
| Walker2d-v5    | trajectory | False      | 4786.1           | 325.7 | 4715.2            | 9.43e-03 | 7458            |
| Walker2d-v5    | trajectory | True       | 4344.3           | 459.2 | 4201.3            | 9.08e-03 | 7053            |

## 0.2 实验图表

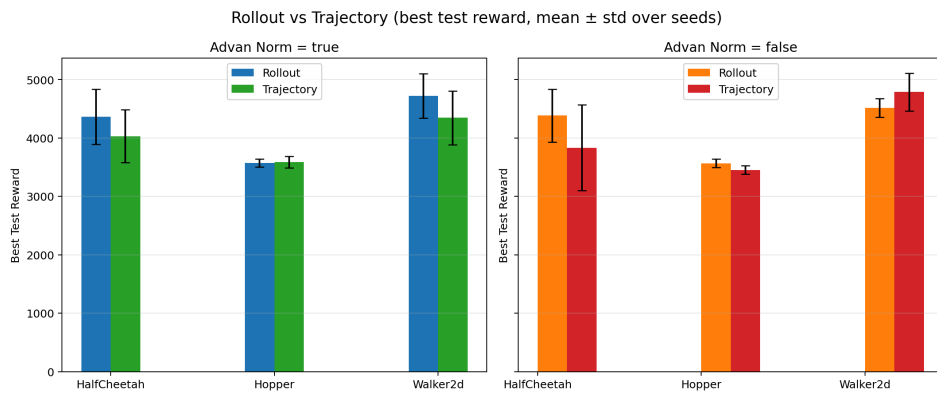
图 1: 样本利用率对比



### Sample Utilization

rollout 每次更新稳定使用 16000 个有效 transition。trajectory 模式由于 episode 提前终止和 padding，有效 transition 数更少，其中 Hopper-v5 差异最明显。

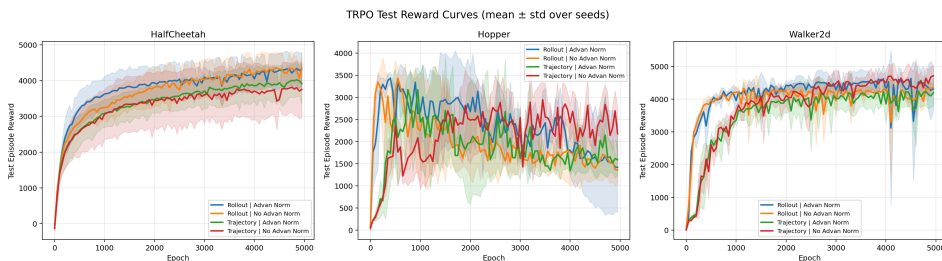
图 2：Rollout 与 Trajectory 的 best reward 对比



### Mode Comparison

整体上 rollout 的平均表现略强，但单个最佳配置并不总是 rollout。例如 Hopper-v5 的最佳配置为 trajectory + advan\_norm=true，Walker2d-v5 的最佳配置为 trajectory + advan\_norm=false。

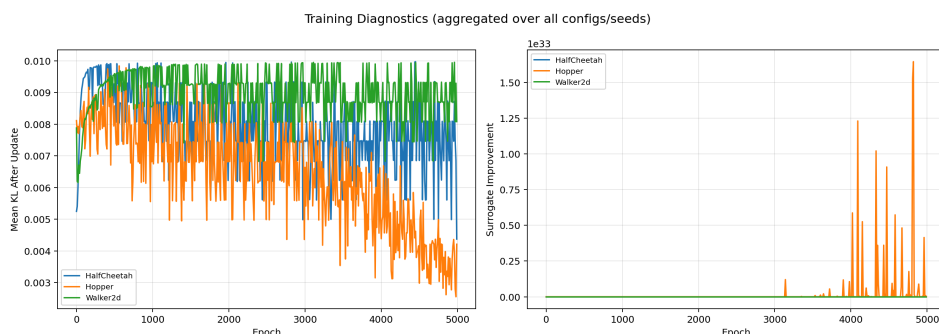
图 3：不同配置的测试回报曲线



### Learning Curves

从曲线看，HalfCheetah-v5 收敛较平滑，Walker2d-v5 最终表现较好，Hopper-v5 波动最大且后期存在明显下降。

图 4：KL 与 Surrogate Objective 诊断曲线



## Training Diagnostics

KL 基本被控制在 0.01 以下，说明 trust region 约束整体生效。但 Hopper-v5 的 surrogate improvement 后期出现异常尖峰，说明该环境下训练存在较强不稳定性。

# 1. 代码补全部分问题回答

## 1.1 collect\_trajectories 实现部分

### 1.1.1 collect\_trajectories 与普通 rollout 采样方式相比，数据收集的停止条件分别是什么？

普通 `rollout` 采样方式通常以固定 transition 数作为停止条件。本实验中 rollout 模式下每次策略更新实际使用 16000 个 transition。即使某个环境中的 episode 提前结束，也会 reset 环境并继续采样，直到 buffer 中 transition 数达到要求。

`collect_trajectories` 则以轨迹为单位进行采样。每条 trajectory 会在环境返回 `done` / `truncated` 或达到最大轨迹长度时结束。采样过程通常在收集满指定数量的 trajectory 后停止，而不是简单按照固定 transition 数停止。

因此，两者区别是：

| 采样方式       | 停止条件            | 数据特点                       |
|------------|-----------------|----------------------------|
| rollout    | 固定 transition 数 | 样本数稳定，可能跨多个 episode        |
| trajectory | 固定轨迹条数或完整轨迹结构   | 保留 episode 边界，可能产生 padding |

### 1.1.2 trajectory 采样模式下每条轨迹需要保存哪些信息？这些信息在后续 TRPO 更新中有什么作用？

每条 trajectory 需要保存以下信息：

| 信息                                         | 作用                                                         |
|--------------------------------------------|------------------------------------------------------------|
| <code>states</code>                        | 输入 actor 和 critic，用于计算 value、log probability、KL divergence |
| <code>actions</code>                       | 用于重新计算当前策略下的 log probability                               |
| <code>rewards</code>                       | 用于计算 TD residual、GAE 和 return                              |
| <code>done</code> / <code>terminals</code> | 判断 episode 是否结束，防止 GAE 和 return 跨越终止边界                     |
| <code>values</code>                        | 用于计算 advantage 和 return                                    |
| <code>old_log_probs</code>                 | 固定旧策略概率，用于计算 importance sampling ratio                     |
| <code>masks</code>                         | 区分真实 transition 与 padding transition                       |
| <code>next_states</code>                   | 用于 bootstrap 和 TD error 计算                                 |

在 TRPO 更新中，`old_log_probs` 和 `actions` 用于构造 surrogate objective；`states` 用于计算当前策略分布和 value；`rewards`、`values`、`done`、`masks` 用于计算 advantage 和 return。

### 1.1.3 当环境返回终止信号时，轨迹如何结束？未达到最大长度的部分如何处理？

当环境返回 `done` 或 `truncated` 时，该 trajectory 结束。如果此时轨迹长度还没有达到最大长度，则后续位置需要进行 padding。

padding 部分不是真实环境交互数据，因此应设置：

```
mask = 0
```

这些 padding 位置不能参与：

- return 计算；
- advantage 计算；
- policy loss；
- value loss；
- reward 统计；
- 样本利用率统计。

有效 transition 数应通过 `sum(masks)` 统计。

### 1.1.4 TrajectoryRollout 中的 masks 变量有什么作用？如果不使用 masks，计算 return 或 advantage 时可能出现什么问题？

`masks` 用于区分真实 transition 和 padding transition：

```
mask = 1: 真实 transition
mask = 0: padding transition
```

如果不使用 `masks`，会出现以下问题：

1. padding 的零 reward 会被误认为真实 reward；
2. padding state 会被输入 critic；
3. padding action 会参与 policy loss；
4. GAE 反向递推可能跨到 padding 区域；
5. return 和 advantage 会产生偏差；
6. value function 会拟合无意义目标；
7. 样本利用率统计会虚高。

因此，在 trajectory 模式下，`masks` 是保证 return、advantage 和 loss 正确计算的关键变量。

### 1.1.5 collect\_trajectories 在收集完一批轨迹后为什么需要重新初始化环境状态？

收集完一批 trajectory 后，不同并行环境可能处于不同状态：有的刚刚终止，有的处于 episode 中间，有的达到 TimeLimit。重新初始化环境可以保证下一批 trajectory 从干净的初始状态开始，避免上一批未保存的残余状态影响下一批采样。

这不会改变 TRPO 的 on-policy 特性，因为 on-policy 的关键在于：用于更新的数据是否由当前策略采样。只要每批 trajectory 来自当前策略，并且只用于当前这次更新，仍然属于 on-policy 训练。

## 1.2 \_update\_policy 实现部分

### 1.2.1 Theorem 1 中每一项的实际意义是什么？在什么情况下可以保证策略单调改进？

TRPO 的理论下界可以概括为：

```
eta(pi_new) >= L_pi_old(pi_new) - penalty(pi_old, pi_new)
```

其中：

| 项                                    | 含义                                  |
|--------------------------------------|-------------------------------------|
| <code>eta(pi_new)</code>             | 新策略的真实期望回报                          |
| <code>L_pi_old(pi_new)</code>        | 在旧策略状态分布下估计的新策略 surrogate objective |
| <code>penalty(pi_old, pi_new)</code> | 新旧策略差异过大导致的惩罚项                      |

该公式说明：如果新策略能够提高 surrogate objective，并且新旧策略之间的距离足够小，那么新策略的真实性能下界也会提高。因此，TRPO 的设计思路是：在限制策略变化幅度的前提下，最大化 surrogate objective。

实际优化形式可以写成：

```
maximize    L_pi_old(pi_new)
subject to  D_KL(pi_old || pi_new) <= delta
```

也就是说，TRPO 不是无限地沿着策略梯度方向更新，而是在 trust region 内进行更新。

## 1.2.2 与 Approximately Optimal Approximate Reinforcement Learning 的 Theorem 4.1 相比，TRPO 的分析处理有什么不同？

早期结果主要分析混合策略形式：

```
pi_new = (1 - alpha) * pi_old + alpha * pi'
```

这种形式依赖显式混合系数，只适合特定的混合策略类。TRPO 的处理方式更一般，它不要求新策略一定由旧策略和候选策略线性混合，而是通过新旧策略之间的距离来控制状态访问分布的变化。

因此，TRPO 可以推广到一般随机策略类，尤其适合神经网络参数化的连续动作策略。

## 1.2.3 TRPO 中策略更新的 surrogate objective 在实际算法中如何设计？

实际算法中使用重要性采样比率构造 surrogate objective：

```
ratio = exp(logp_new(a|s) - logp_old(a|s))
L = mean(ratio * advantage)
```

其中：

- `logp_old` 是采样时旧策略对动作的 log probability；
- `logp_new` 是当前策略重新计算的 log probability；
- `advantage` 是由旧策略采样数据计算得到的优势函数；
- `ratio` 表示新策略相对于旧策略选择该动作的概率变化。

在实现中，如果优化器默认最小化 loss，通常写成：

```
loss_pi = -mean(ratio * advantage)
```

## 1.2.4 为什么 TRPO 约束 KL divergence 而不是定理中的 TV 距离？为什么需要 line search？

理论中使用 TV distance 便于给出性能界，但在神经网络策略中，TV distance 不容易稳定计算和求导。KL divergence 对高斯策略更容易计算，并且可以通过自动微分得到 Hessian-vector product，因此更适合实际实现。

在旧参数附近，KL divergence 可以进行二阶近似：

```
D_KL(pi_old || pi_theta) ≈ 1/2 * delta_theta^T H delta_theta
```

其中 `H` 可以近似看作 Fisher 信息矩阵。基于这个二阶近似，TRPO 可以通过 conjugate gradient 求自然梯度方向。

但神经网络是非线性的，有限 batch 也会带来估计误差，所以理论上计算出的步长不一定真的满足 KL 约束。line search 的作用是检查真实更新后是否满足：

1. 实际 KL 不超过阈值；
2. surrogate objective 有提升；
3. 参数没有出现 NaN 或 Inf。

如果不满足，就缩小步长；如果所有候选步长都失败，则回滚参数。

## 2. 探索部分问题回答

### 2.1 在相同超参数设置下，三个环境的训练难度是否一致？请结合 reward 曲线分析。

三个环境训练难度不一致。

从 reward 曲线看，`HalfCheetah-v5` 的训练最平滑，四种配置都能稳定上升到 3500 到 4400 左右，说明该环境的 reward 较密集，动力学相对稳定。

`Hopper-v5` 的训练最不稳定。虽然 best reward 可以达到约 3500，但 final reward 明显下降，尤其 rollout 配置后期回落明显。这说明 Hopper 容易受到策略退化、提前终止或 value 估计误差的影响。

`Walker2d-v5` 的最终表现最好，多数配置能够达到 4000 以上，其中 `trajectory + advan_norm=false` 的 mean best reward 达到 4786.1，是本实验中的最高结果。

因此，同一组 TRPO 超参数在不同 MuJoCo 环境中的效果并不一致。总体上，HalfCheetah 更稳定，Hopper 更难稳定训练，Walker2d 能达到较高最终性能。

### 2.2 哪个环境中 trajectory 与 rollout 的样本利用率差异最明显？可能原因是什么？

样本利用率差异最明显的是 `Hopper-v5`。

本实验中 rollout 每次更新稳定使用 16000 个有效 transition，而 trajectory 模式下各环境有效 transition 数如下：

| Env            | Trajectory Effective Transitions |
|----------------|----------------------------------|
| HalfCheetah-v5 | 10000                            |
| Hopper-v5      | 约 4142                           |
| Walker2d-v5    | 约 7256                           |

`Hopper-v5` 的 trajectory 有效 transition 数最低，说明它产生了最多 padding。主要原因是 Hopper 容易提前摔倒，episode 长度短且波动大。trajectory 模式需要将不同长度轨迹 padding 到统一长度，因此提前终止越多，`mask=0` 的无效位置就越多，样本利用率越低。

### 2.3 trajectory 模式和 rollout 模式下 advantage 与 return 的计算方式有什么不同？

trajectory 模式按完整 episode 或 trajectory 组织数据，必须处理变长轨迹和 padding。因此 GAE 递推时既要考虑 episode 是否结束，也要考虑下一步是否为有效样本：

```
non_terminal_t = (1 - done_t) * next_mask_t
```

rollout 模式按固定 transition 数组织数据，通常不需要 padding，但一个 buffer 可能包含多个 episode。因此它主要通过 `done` 来断开 GAE：

```
non_terminal_t = 1 - done_t
```

此外，rollout buffer 的末尾可能不是 terminal，因此需要使用最后一个状态的 value 进行 bootstrap；trajectory 模式中每条轨迹通常在 done、truncated 或最大长度处结束，且 padding 部分通过 mask 排除。

## 2.1 Advantage Normalization 的影响

### 2.1.1 `advan_norm=true` 和 `advan_norm=false` 的平均回报曲线有何差异？

从实验结果看，advantage normalization 的影响较小但具有环境依赖性。

整体平均上：

```
advan_norm=true: 4102.8
advan_norm=false: 4089.7
```

`advan_norm=true` 略高于 `advan_norm=false`，但差距不大。

分环境看：

- `HalfCheetah-v5`：rollout 下 `advan_norm=false` 略好，trajectory 下 `advan_norm=true` 更好；
- `Hopper-v5`：两种采样方式下 `advan_norm=true` 的 best reward 都略高；
- `Walker2d-v5`：rollout 下 `advan_norm=true` 更好，但 trajectory 下 `advan_norm=false` 明显更好。

因此，advantage normalization 并不是在所有环境和采样方式下都带来性能提升。

## 2.1.2 加入 advantage normalization 后，训练过程是否更加稳定？请结合 reward 曲线、KL divergence 或 surrogate objective 分析。

从 KL 曲线看，所有环境的 mean KL 基本被控制在 0.01 以下，说明 TRPO 的 KL 约束整体有效。advantage normalization 对 KL 稳定性有一定帮助，但不是决定性因素。

从 reward 曲线看，`Hopper-v5` 的波动明显大于其他环境，即使开启 advantage normalization，也仍然存在后期 reward 下降的问题。从 surrogate objective 诊断图看，Hopper 在后期出现了非常大的异常尖峰，说明其训练过程中存在数值不稳定或 surrogate 估计异常。

因此，advantage normalization 能缓解 advantage 尺度变化带来的问题，但不能完全解决 Hopper 中的训练不稳定。

## 2.1.3 在选择的测试环境中，advantage normalization 是否总是带来性能提升？如果不是，可能原因是什么？

不是。

最明显的例子是 `Walker2d-v5` 的 trajectory 模式：

| Config                                     | Mean Best Reward |
|--------------------------------------------|------------------|
| trajectory + <code>advan_norm=false</code> | 4786.1           |
| trajectory + <code>advan_norm=true</code>  | 4344.3           |

`advan_norm=false` 明显更好。

可能原因包括：

1. 原始 advantage 的尺度本身包含有用的回报强弱信息；
2. normalization 改变了不同轨迹之间的相对优势幅度；
3. trajectory 模式有效样本数较少，均值和方差估计更容易受 seed 影响；
4. TRPO 已经通过 KL 约束和 line search 控制了更新幅度，advantage normalization 的额外收益有限；
5. value function 的估计误差无法仅靠 advantage normalization 消除。

## 2.2 Trajectory 与 Rollout 的样本利用率比较

### 2.2.1 rollout 模式下每次策略更新实际使用了多少个 transition？trajectory 模式下每次策略更新又实际使用了多少个有效 transition？

本实验中，rollout 模式每次策略更新实际使用：

16000 transitions

trajectory 模式的有效 transition 数由 `sum(masks)` 决定，不同环境结果如下：

| Env            | Trajectory Effective Transitions |
|----------------|----------------------------------|
| HalfCheetah-v5 | 10000                            |
| Hopper-v5      | 约 4142                           |
| Walker2d-v5    | 约 7256                           |

## 2.2.2 对比 trajectory 与 rollout 两种模式，哪一种样本利用率更高？

rollout 模式样本利用率更高。

在三个环境中，rollout 都稳定使用 16000 个有效 transition，而 trajectory 模式因为 episode 提前结束和 padding，实际有效 transition 数明显更少。尤其在 Hopper 中，trajectory 有效 transition 只有约 4142，远低于 rollout 的 16000。

## 2.2.3 样本利用率更高是否一定意味着训练效果更好？请结合实验曲线分析。

不一定。

从实验结果看：

- `HalfCheetah-v5` 中 rollout 的表现更好，样本利用率优势与 reward 表现一致；
- `Hopper-v5` 中 rollout 整体 best reward 略高，但单个最佳配置是 `trajectory + advan_norm=true`；
- `Walker2d-v5` 中最高 mean best reward 来自 `trajectory + advan_norm=false`，尽管 trajectory 的有效 transition 数低于 rollout。

因此，样本利用率高通常有利于训练效率，但不必然带来更高 reward。训练效果还取决于 advantage 估计质量、critic 误差、episode 结构、环境动力学以及 KL 和 line search 的稳定性。

## 3. 总结与反思

### 3.1 在本项目中，`collect_trajectories` 的实现是否会改变 TRPO 的 on-policy 特性？

不会。

TRPO 的 on-policy 特性取决于数据是否由当前策略采样，并且是否只用于当前这次策略更新。`collect_trajectories` 只是改变数据组织方式和采样停止条件，不改变数据来源。

只要每批 trajectory 来自当前策略，并且更新后清空 buffer，不反复使用旧数据，就不会破坏 on-policy 特性。

### 3.2 如果只能保留一种采样方式，会选择 trajectory 还是 rollout？请结合样本利用率和训练性能说明理由。

如果从工程训练效率出发，我更倾向于保留 rollout。

理由是：

1. rollout 每次更新稳定使用 16000 个有效 transition；
2. 不需要 padding，样本利用率更高；
3. 实现更简单，不需要额外处理 masks；
4. HalfCheetah 和 Hopper 中 rollout 的整体平均表现更好；
5. Walker2d 中 trajectory 的单个最佳结果更高，但 rollout 的平均表现也很接近。

不过，如果实验重点是分析完整 episode 结构、提前终止和 masks 机制，则 trajectory 更有分析价值。综合本次实验，rollout 更适合作为默认训练方式，trajectory 更适合作为对照实验和机制分析。

### 3.3 完成实验报告后，对 TRPO 算法的新认识

通过实验可以看到，TRPO 的 KL 约束确实能较好地限制策略更新幅度，但这并不意味着训练过程一定完全稳定。Hopper 中 reward 后期下降和 surrogate objective 异常尖峰说明，TRPO 的稳定性还依赖 advantage 估计、value function 拟合和环境本身的动力学难度。

因此，TRPO 的稳定训练不仅需要正确实现 KL 约束和 line search，也需要保证 return / advantage 计算、mask 处理和 critic 训练足够可靠。

最大的收获感觉是在强化学习方面理论和实践可能产生一些出入。比如理论更优的 TRPO 算法在实际工程中速度和效果都不如 PPO，这是比较反直觉的，让我感觉模型的训练更像炼丹，一些小小的改进都可能带来巨大的提升。以后也会对有疑虑的地方展开更多探索实验。



---

### 3.4 对本次算法实现和实验引导的建议

感觉可以增加更多工程上能提升实验效果的tricks介绍和尝试，

---

## 4. 总体结论

本实验完成了 TRPO 在三个 MuJoCo v5 环境下的比较测试。主要结论如下：

1. 三个环境训练难度不同，HalfCheetah 最平滑，Hopper 最不稳定，Walker2d 最终性能最高；
2. rollout 的样本利用率显著高于 trajectory；
3. 样本利用率更高不一定带来最高 reward，Walker2d 中 trajectory + advan\_norm=false 获得最佳结果；
4. advantage normalization 整体平均略有提升，但不是所有环境和采样方式下都有效；
5. KL 约束整体生效，但 Hopper 的 surrogate objective 存在异常尖峰，说明仍需进一步稳定训练过程；
6. 如果只保留一种采样方式，rollout 更适合作为默认训练方案，trajectory 更适合作为分析完整轨迹结构的实验模式。